ASSIGNMENT 1

Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

Course: CSA09 – Java Fundamentals and Object-Oriented Programming

Student Name: P. Rahul Reddy
Register Number: 192324214

Repository and Execution Details

Repository name: CSA09-192324214-Assignment1

Repository URL: https://github.com/PRAHULREDD/CSA09-192324214-Assignment1.git

Environment: Windows PowerShell; OpenJDK 21.0.12.1 LTS.

Command used: .\run-all.ps1

The run-all.ps1 script compiled and ran each task in a separate build folder.

Contents

1. Task 1 – Understand the Problem and Identify the Objects
2. Task 2 – Choose the Data Representation
3. Task 3 – Design the Control Logic
4. Task 4 – Design Classes, Objects and Constructors
5. Task 5 – Apply Static Members and the Object Life Cycle
6. Task 6 – Design the Inheritance Hierarchy
7. Task 7 – Override and Specialise Behaviour
8. Task 8 – Apply Polymorphism
9. Task 9 – Introduce Abstraction
10. Task 10 – Enforce Encapsulation through Packages and Access Modifiers
11. Task 11 – Implement, Test and Defend the Design
12. Task 12 – Re-evaluate and Adapt the Design

Task 1 – Understand the Problem and Identify the Objects

Purpose

The system manages vehicles entering and leaving a multi-level campus parking facility. It allocates a suitable free slot, issues a unique parking ticket, records entry time, calculates the parking fee at exit, and releases the slot. Students and staff with valid monthly passes park free; visitors pay the applicable fee. Electric vehicles may use charging slots with a separate charging fee.

Inputs and outputs

Inputs include registration number, vehicle category, user/pass category, entry and exit times, requested slot or floor where applicable, and energy consumed for EV charging. Outputs include slot allocation result, ticket number, occupancy state, parking fee, charging fee, exit confirmation, and daily revenue.

Candidate classes and responsibilities

Vehicle represents a registered vehicle and its common state. TwoWheeler, Car, ElectricCar and later E-bike and DeliveryVan represent specialised categories. ParkingSlot represents one physical space and whether it is occupied. ParkingFacility represents floors and slots and allocates/releases spaces. ParkingTicket records the vehicle, unique ticket identifier, and entry time. ParkingService or ParkingSystem coordinates entry, exit, active tickets, and revenue. Pass represents a parking entitlement and validity. Chargeable represents the capability to incur EV charging charges.

Nouns-to-classes and verbs-to-methods

Candidate nouns include vehicle, slot, floor, ticket, pass, facility, fee, and charging bay. Candidate verbs become operations such as allocate(), release(), enter(), exit(), calculateFee(), chargingFee(), isValid(), and findTicket(). This mapping helps keep responsibilities explicit rather than placing all behaviour in one large class.

Business rules and assumptions

A slot cannot hold more than one vehicle. A vehicle cannot be parked twice at the same time under the same registration. Ticket identifiers must be unique and not reused during the system lifetime. Invalid floor/slot coordinates must be rejected. Fees cannot be negative. A valid pass must be checked for its applicable period and category. The system uses a jagged array where floor capacities differ. Unless otherwise specified, a started hour is charged in full in the supplied FeeDemo example.

Alternative: objects vs parallel arrays

An object list stores each vehicle's related state and behaviour together. Parallel arrays such as regNo[], type[], and entryTime[] store the same logical record across separate arrays. With n vehicles, a linear search by registration is O(n) in either representation. Object storage uses O(n) references plus per-object fields; parallel arrays use O(n) entries per attribute. Adding an attribute to parallel arrays requires another array and careful index synchronisation; an object design usually adds a field and updates constructors/accessors. Objects improve cohesion and maintainability, but do not automatically make linear search faster.

Task 2 – Choose the Data Representation

Field types

Use String for registration numbers because they contain letters and leading zeros. Use int for floor index, slot index, and duration in minutes where limits are modest. Use LocalDateTime or a suitable time type for entry/exit timestamps. Use long for money in paise or BigDecimal for currency calculations; double is used by the provided FeeDemo example but is less suitable for production accounting.

Floor and slot representation

A jagged array such as String[][] slots allocates one array per floor and supports unequal floor capacities. The execution demonstration used capacities [20, 35, 50]. A rectangular array wastes spaces or imposes a common floor capacity. A one-dimensional flattened array is convenient only when a reliable mapping from floor/slot to a single index is maintained; for unequal floor sizes, prefix offsets are needed. ArrayList<String[]> also supports variable capacities, with each element representing one floor.

Casting and integer division risk

Integer division truncates: 61 / 60 evaluates to 1, so charging one hourly rate would undercharge a stay of 61 minutes if the rule is every started hour. The correct ceiling for non-negative minutes using integer arithmetic is (minutes + 59) / 60. For 61 minutes this gives 2 hours. The user's Task02 output reports a correct fee of Rs 60.0 and an incorrect integer-division fee of Rs 30.0, illustrating the error.

Input handling

Scanner is appropriate for an interactive console program; command-line arguments are useful for scripted test runs. Input should be validated before modifying facility state.

Alternative representation and complexity

For F floors and S total slots, scanning all slots takes O(S) time in the worst case. A flattened array also needs O(S) space and can locate a known index in O(1), but finding a free slot by scanning remains O(S). ArrayList<String[]> stores the same O(S) slots and handles unequal capacities naturally, with indexed access O(1); scanning for a free slot remains O(S). A jagged array requires O(S) slot references plus O(F) floor-array references.

Task 3 – Design the Control Logic

Allocation pseudocode

```
ALGORITHM Allocate(vehicleNo):
1. IF vehicleNo is null or invalid, RETURN failure.
2. FOR each floor f from 0 to numberOfFloors - 1:
3.   FOR each slot s from 0 to slots[f].length - 1:
4.     IF slots[f][s] is null:
5.       slots[f][s] = vehicleNo
6.       RETURN {f, s}
7. RETURN null (facility full).
```

Fee pseudocode

```
ALGORITHM CalculateFee(minutes, hourlyRate, dailyMaximum):
1. IF minutes < 0 OR hourlyRate < 0, RETURN validation error.
2. IF minutes = 0, billableHours = 0.
3. ELSE billableHours = (minutes + 59) DIV 60.
4. fee = billableHours × hourlyRate.
5. IF dailyMaximum is configured, fee = MIN(fee, dailyMaximum).
6. RETURN fee.
```

Control statements

Nested loops search floor-by-floor and slot-by-slot. return stops immediately when a slot is found. If no slot is available after both loops, return null. release() validates both indices and verifies that the slot is occupied before clearing it. continue can skip unsuitable slots; break can stop a loop when a match is found, although returning the coordinates is clearer here.

Actual execution output

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Alternative: scan vs free-slot stack

A full scan has O(S) worst-case allocation and O(1) extra auxiliary space. A stack/queue of free slot coordinates can provide O(1) amortised allocation and release, assuming the structure is kept consistent. The free-slot structure requires O(S) additional storage. Per-floor counts can quickly identify whether a floor has free capacity but do not alone identify the exact free slot.

Task 4 – Design Classes, Objects and Constructors

Constructors and this

A default constructor can create an object with safe defaults; a parameterised constructor can initialise required fields; overloaded constructors support different valid creation paths. this.field distinguishes an instance field from a parameter of the same name. Constructor chaining with this(...) centralises validation and avoids duplicated initialisation. A copy constructor is useful only if the program needs a deliberate duplicate/snapshot object; a duplicate active ticket should not accidentally reuse a unique ticket number.

Memory and references

Local reference variables are typically held in stack frames, while objects are allocated on the heap. A reference points to an object; assigning a reference copies the reference value rather than cloning the object. An object becomes eligible for garbage collection when it is no longer reachable from live references, though collection timing is not guaranteed. The user's Task04 output demonstrates ticket objects and states that references point to heap objects.

Actual execution output

```
Ticket{1, UNKNOWN, 2026-10-20T19:45:56.907464100}
Ticket{2, TN01AB1111, 2026-10-20T19:45:56.907464100}
Ticket{3, TN01AB1111, 2026-10-20T19:45:56.907464100}
Reference variables point to objects; objects are allocated on the heap.
```

Alternative: setters or factories

A default constructor followed by setters can create temporarily invalid objects unless validation is enforced after every setter. Constructors can guarantee required fields at creation. Static factory methods can use meaningful names, validate input, and return cached or specialised instances if needed. Creating n objects requires O(n) total time and O(n) space for the objects in all approaches; differences are primarily validation safety, readability and maintainability rather than asymptotic complexity.

Task 5 – Apply Static Members and the Object Life Cycle

Static vs instance state

A static ticket counter belongs to the class and is shared by all instances of that class. Instance state such as the occupancy of the north or south facility belongs to each ParkingFacility object. Static methods are appropriate for operations that do not depend on a particular instance, such as a pure conversion helper; a static initialisation block is appropriate only for one-time class setup.

Lifecycle and passing objects

When a vehicle exits, its active ticket should be removed from the active-ticket structure and its slot released. The ticket is eligible for garbage collection when no remaining live references point to it. Methods receive object references by value and can return references to objects; changing an object's fields through a reference is visible to other references to that same object.

Actual execution output

```
North occupancy=1, ticket=1
South occupancy=2, ticket=2
Active tickets=1; next shared ID=3
```

Alternative: shared vs per-facility counter

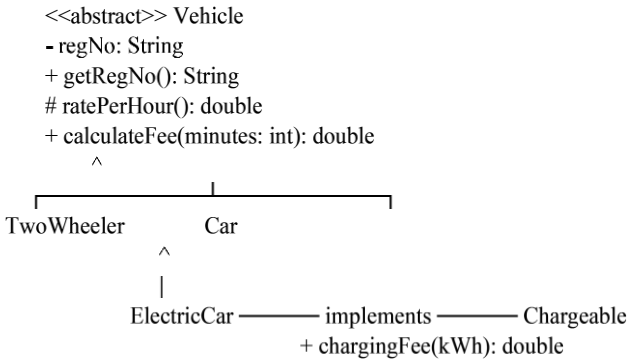
A static counter provides one sequence across all facilities in the same running process, using O(1) counter storage independent of the number k of facilities and n tickets. An instance counter provides a separate sequence per facility, using O(k) counter storage. If uniqueness must be global across processes or persisted across restarts, a database sequence or UUID is more reliable than an in-memory counter.

Task 6 – Design the Inheritance Hierarchy

Hierarchy

Vehicle is the superclass. TwoWheeler and Car are subclasses; ElectricCar extends Car and implements Chargeable. This is hierarchical inheritance with a multilevel path Vehicle → Car → ElectricCar. Common state such as registration number belongs in Vehicle, while rate or specialised capabilities belong in subclasses or interfaces.

UML class diagram (text form)



super and constructor trace

The constructor trace observed in Task06 is:
Vehicle constructor
Car constructor
ElectricCar constructor
Vehicle constructor
The first three lines show superclass-to-subclass constructor execution when creating an ElectricCar. The final line corresponds to another vehicle object created by the demo. super(...) invokes the parent constructor and must be the first constructor statement in the usual Java constructor form.

Actual execution output

```
Vehicle constructor
Car constructor
ElectricCar constructor
Vehicle constructor
EV rate=30.0, bike rate=10.0
```

Alternative: inheritance vs composition

With inheritance, adding a vehicle type typically means adding a subclass and overriding the appropriate behaviour. With composition, Vehicle can contain a VehicleType or PricingRule object. Both can compute a fee in O(1) when the selected rate/rule is directly available. Composition can make changing pricing policies at runtime easier; inheritance expresses stable is-a relationships and may create many subclasses if type and pricing variations multiply. Per-vehicle storage is O(1) in either design, with composition adding a reference to the strategy/type object.

Task 7 – Override and Specialise Behaviour

Overriding rules

An overriding method uses the same signature and a compatible return type; its access cannot be more restrictive than the overridden method. @Override asks the compiler to verify the intent. Dynamic dispatch selects the implementation based on the runtime object type. final variables cannot be reassigned after initialisation, final methods cannot be overridden, and final classes cannot be subclassed.

Object methods

toString() should provide useful diagnostic text. equals() should define a consistent equality rule, such as registration-number equality if registrations uniquely identify vehicles. hashCode() must return the same value for objects considered equal. The policy should be consistent with case sensitivity and registration normalisation.

Actual execution output

```
Car[TN01AB1111] fee for 61 min=Rs 60.0
Bike[TN02CD2222] fee for 61 min=Rs 20.0
Same registration equals=true
```

Alternative: overriding vs switch

A switch on a type field centralises behaviour but must be updated when a new type is added, and can spread type checks across the system. Overriding lets each subclass own its behaviour. With a fixed type, either approach can calculate a fee in O(1); the main difference is coupling and extension cost rather than asymptotic runtime. A switch typically changes at least the central fee method when adding a category.

Task 8 – Apply Polymorphism

Compile-time and runtime polymorphism

Overloading provides methods with the same name but different parameter lists, for example calculateFee(int minutes) and calculateFee(LocalTime entry, LocalTime exit). The compiler selects the overload from the declared argument types. Runtime polymorphism occurs when a Vehicle reference points to a Car or TwoWheeler and the overridden ratePerHour() implementation is selected from the actual object type.

Dispatch trace

Declared reference type: Vehicle. Actual object type: ElectricCar. Call: calculateFee(130). Vehicle.calculateFee() invokes ratePerHour(); runtime dispatch selects Car.ratePerHour(), returning Rs 30/hour. Since 130 minutes spans three started hours, the parking fee is 3 × Rs 30 = Rs 90. The Chargeable interface check adds 10 kWh × Rs 12/kWh = Rs 120, giving Rs 210, which matches the supplied FeeDemo output.

Actual FeeDemo output

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
truc
```

Equality result

The last line prints true because the supplied equals() implementation compares registration numbers and ignores concrete subclass type. A Car and ElectricCar with the same registration are therefore equal under that implementation. This is correct only if registration number alone defines identity and registrations are normalised consistently. In an operational system, duplicate active registration should still be rejected.

Alternative: instanceof chain

An instanceof/casting chain checks the runtime category and can require up to k checks for k vehicle categories in a worst-case ordered chain, giving O(k) type-dispatch work. Virtual method dispatch is conceptually O(1) for selecting the implementation. More importantly, a long conditional chain becomes more difficult to extend and test as categories grow.

Task 9 – Introduce Abstraction

Abstract class and interface

Vehicle is abstract because a generic vehicle has shared state and common fee logic but no single concrete hourly rate. Chargeable is an interface because charging is a capability that may apply to some vehicles without defining their primary inheritance. A class can implement multiple interfaces, enabling a vehicle to be both chargeable and eligible for another independent capability.

Default and static interface methods

A default interface method can provide shared optional behaviour to implementers. A static interface method can provide a utility related to the capability. These should be used when they clarify the abstraction rather than simply adding methods for demonstration.

Actual execution output

```
Abstract type=Car
Charging bay required=true, total=Rs 210.0
```

Alternative swapped design

If Vehicle became an interface, it could declare behaviour and provide default methods, but it could not hold ordinary per-instance fields with constructor initialisation in the same way as an abstract class. A class can implement multiple interfaces but extend only one class, so turning Chargeable into an abstract class would prevent ElectricCar from extending both Car and that charging base class. Per-call dispatch remains effectively constant-time in either design; the main trade-off is state reuse and inheritance flexibility.

Task 10 – Enforce Encapsulation through Packages and Access Modifiers

Package layout

The source layout uses parking.model for domain objects, parking.service for business operations and parking.app for the entry point. This separates data models, service logic and application startup.

Access and validation

Fields should generally be private. Public methods expose supported operations; protected members are appropriate only for subclass extension; package-private members are appropriate for tightly related package internals. Constructors and setters should reject invalid registration numbers, negative durations, invalid rates and invalid slot coordinates. Validation should occur at the object/service boundary rather than relying on every caller to remember the same checks.

Actual execution output

```
Ticket=1, vehicle=TN01AB1111
Exited=true, active=0
```

Wrappers and alternative public fields

Autoboxing converts between primitive values and wrapper objects, for example int and Integer. Integer references should not generally be compared with == to compare numeric values because == may compare object identity; use equals() or unbox carefully. Public fields allow callers to bypass validation. Validation per update is O(1), but if duplicated in many callers, the total maintenance burden grows with the number of call sites and increases the chance of inconsistent rules.

Task 11 – Implement, Test and Defend the Design

Test evidence from the run

Execution results: Last free slots allocated=true; Full facility returns null=true; Bike fee 60 min=Rs 10.0; Car fee 61 min=Rs 60.0; Invalid slot rejected=true; Duplicate registration rejected=true; Active tickets=1; Hand-calculated sample total=Rs 70.0 (bike Rs10 + car Rs60). These are the results observed in the supplied terminal log; add source-level test details and screenshots.

Test table

Test	Expected result	Actual evidence
Last free slot	Allocation succeeds	Reported true
Facility full	Allocation returns null	Reported true
Bike 60 minutes	Rs 10.00	Rs 10.00 in log
Car 61 minutes	Rs 60.00	Rs 60.00 in log
Invalid slot	Rejected	Reported true
Duplicate registration	Rejected	Reported true
Daily total	Rs 70.00	Rs 70.00 in log
Zero-minute stay	Define and test policy	NOT YET RECORDED
Exact 60-minute boundary	Test started-hour rule	NOT YET RECORDED
Unknown vehicle type	Reject without state mutation	NOT YET RECORDED

Manual revenue calculation

For the demonstrated sample: bike fee Rs 10 + car fee Rs 60 = Rs 70 total. This agrees with the total shown in the test output. Preserve the input scenario and actual output screenshot in the report so the calculation can be reproduced.

Limitation

A simple in-memory implementation loses active parking and ticket data when the program terminates. A production system would need persistent storage, stronger concurrent-allocation protection, audit records and a clearly specified fee/pass policy.

Alternative ticket structures

An array uses O(n) space and O(n) worst-case search by registration. ArrayList also uses O(n) space and O(n) search, but supports easier insertion/removal. HashMap<String, ParkingTicket> uses O(n) space and expected O(1) lookup/insert/remove by registration under normal hashing assumptions, at the cost of hash-table overhead and key uniqueness management.

Task 12 – Re-evaluate and Adapt the Design

Impact analysis

The new requirements add E-bike, DeliveryVan and a one-day visitor pass. The facility grows to five floors with capacities [20, 25, 30, 35, 40] in the test output. The jagged array is suitable because the floors have unequal capacities. Pricing behaviour and pass validity need explicit abstractions so that new categories do not require changing many unrelated classes.

Evaluation and design decision

A design based only on type switches is likely to require modifying central fee logic for each new vehicle. A polymorphic design can add a new subclass, while a PricingRule/strategy abstraction can isolate time-window pricing. The recommended direction is to retain the tested core allocation logic, modify the model/service only where the new requirements demand it, and isolate pricing/pass rules to limit future changes.

Implementation output observed

Floors=5, capacities=[20, 25, 30, 35, 40]
E-bike flat fee=Rs 25.0
Delivery van crossing 09:00-17:00 fee=Rs 150.0
Valid one-day visitor pass fee=Rs 0.0
Active vehicles=0, revenue today=Rs 175.0

Required additional tests

E-bike flat fee; delivery van before 09:00; delivery van at 09:00; delivery van within the window; delivery van at 17:00; delivery van after 17:00; valid visitor pass; expired visitor pass; each of five floor capacities; full facility; slot release and reallocation. Record actual output for each test. The supplied output only demonstrates a van case described as crossing the 09:00–17:00 window; it does not by itself prove all boundary policies.

Evidence-based defence

The execution demonstrates the new categories and five floor capacities. Before making a final claim about extension effort, count the existing source classes that changed when implementing Task12 and list those class names. A precise changed-class count cannot be inferred from console output alone. Trade-offs include additional rule abstractions and tests versus lower coupling when future categories are added.